
Foundations of Deep Learning

Lecture 01

Neural Networks: Introduction

Aurelien Lucchi

Fall 2026

SECTION 1

What's Deep Learning?

Typical classification setting: Given some data $\mathcal{X} \in \mathbb{R}^d$ with some labels, e.g. ± 1 (black or white).

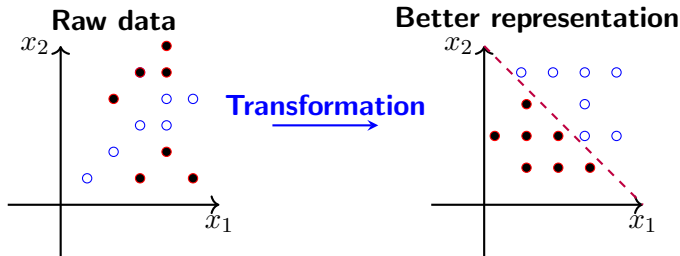
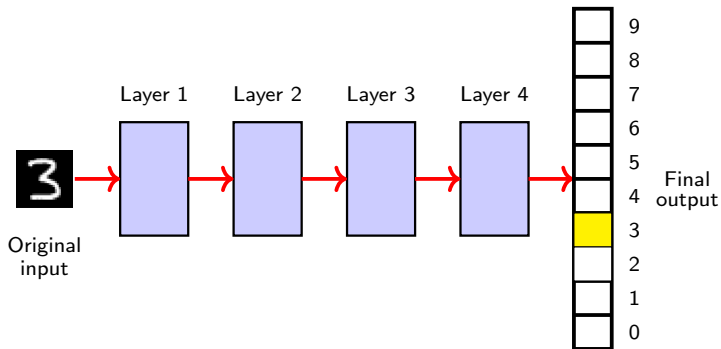


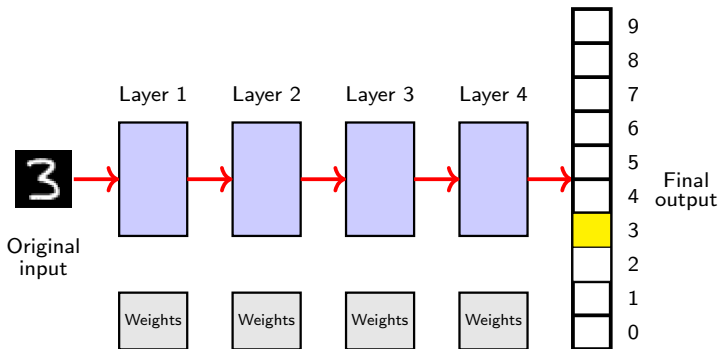
Figure: Left figure shows some raw data that is not linearly separable. Right figure shows transformed data that is linearly separable (as shown by the dotted line that separates the two classes).

Nested representation: Better representations are obtained by passing the raw data through a series of layers that each apply a coordinate change to the data.



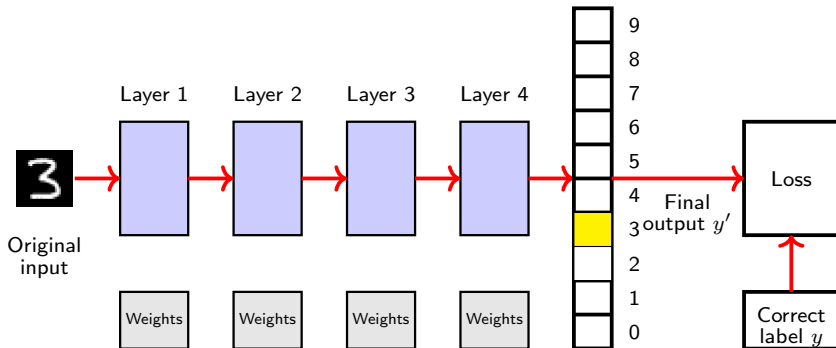
Layer Parameterization

Weights: Each layer has a set of weights that define the transformation applied to the layer input.



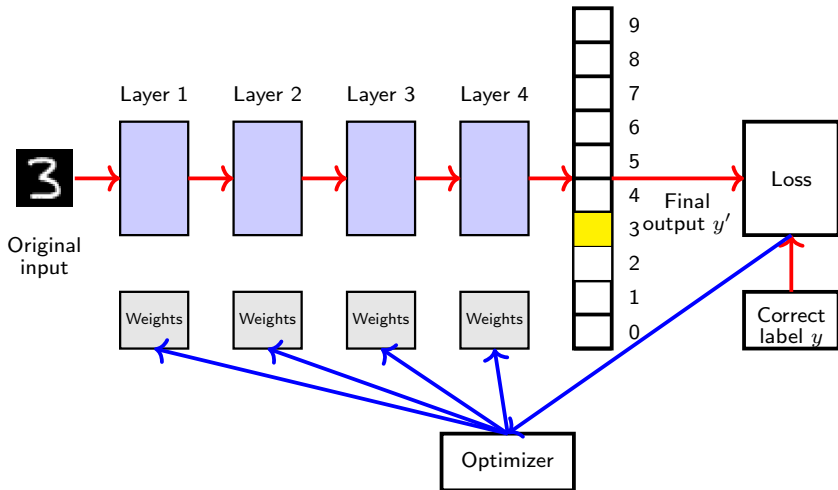
Loss function

The weights are learned by minimizing a loss function.

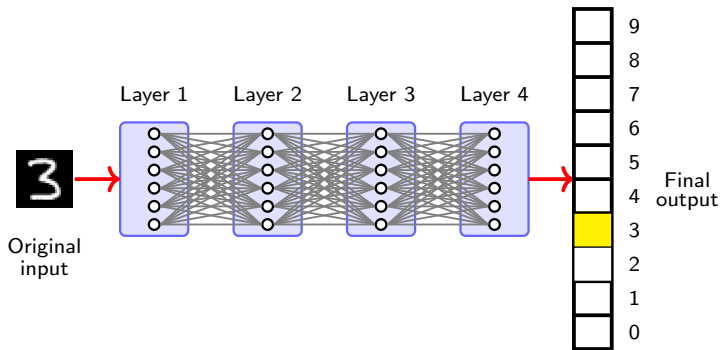


Optimizer

The loss is minimizing using an optimizer, e.g. gradient descent.

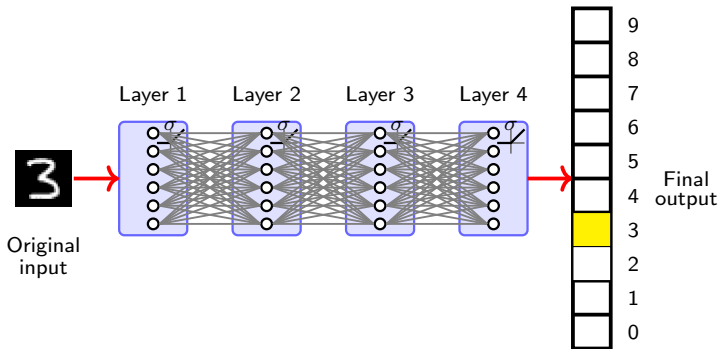


Part 1: Linear Networks



Syllabus for the remainder of this Lecture

Part 2: Adding non-linearities.



SECTION 2

Deep Linear Networks

SECTION 2.1

Basic components: Linear Units

- ▶ Main idea: define a complex system from its elementary building blocks
- ▶ In the case of (deep) neural networks: **computational units** or neurons
- ▶ Complicated functions obtained through **nested composition**



- ▶ Main idea: define a complex system from its elementary building blocks
- ▶ In the case of (deep) neural networks: **computational units** or neurons
- ▶ Complicated functions obtained through **nested composition**
- ▶ Mathematically, such units are – at the highest level of abstraction – real-valued functions



$$f : \mathbb{R}^n \times \mathbb{R}^d \rightarrow \mathbb{R}, \quad (\mathbf{x}, \boldsymbol{\theta}) \mapsto f(\mathbf{x}; \boldsymbol{\theta}).$$

(unit)

Linear Unit

$$g(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{x} \cdot \boldsymbol{\theta}, \quad \mathbf{x} \cdot \boldsymbol{\theta} \equiv \sum_{i=1}^n x_i \theta_i$$

(linear unit)

(output = additively weighted combination of inputs)

Affine Unit

$$g(\mathbf{x}; \boldsymbol{\theta}, b) = \mathbf{x} \cdot \boldsymbol{\theta} + b = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \boldsymbol{\theta} \\ b \end{pmatrix}.$$

(affine unit)

Linear unit defines:

1. **Direction of change** via $\theta / \|\theta\|$
2. **Rate of change** via $\|\theta\|$

Linear unit defines:

1. **Direction of change** via $\boldsymbol{\theta}/\|\boldsymbol{\theta}\|$
2. **Rate of change** via $\|\boldsymbol{\theta}\|$

To create more expressive functions, we can use multiple units arranged in a **linear layer**, resulting in

$$L : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad L(\mathbf{x}; \Theta) = \Theta \mathbf{x},$$

(linear map)

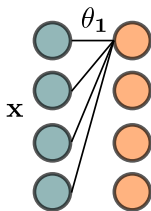
$$\Theta = \begin{pmatrix} \boldsymbol{\theta}_1^\top \\ \vdots \\ \boldsymbol{\theta}_m^\top \end{pmatrix} = \begin{pmatrix} - & \boldsymbol{\theta}_1^\top & - \\ & \vdots & \\ - & \boldsymbol{\theta}_m^\top & - \end{pmatrix}, \quad \Theta \in \mathbb{R}^{m \times n}$$

Note by convention that $\boldsymbol{\theta}_i$ is a column vector, so $\boldsymbol{\theta}_i^\top$ is a row vector.

Note that the layer weight matrix Θ is related to the parameters associated with single units via

$$g_j(\mathbf{x}; \boldsymbol{\theta}_j) = \mathbf{x} \cdot \boldsymbol{\theta}_j, \quad \Theta = \begin{pmatrix} \boldsymbol{\theta}_1^\top \\ \vdots \\ \boldsymbol{\theta}_m^\top \end{pmatrix}, \quad \text{so that} \quad \Theta \mathbf{x} = \begin{pmatrix} \mathbf{x} \cdot \boldsymbol{\theta}_1 \\ \vdots \\ \mathbf{x} \cdot \boldsymbol{\theta}_m \end{pmatrix}.$$

- ▶ increasing layer width m : extract many features simultaneously
- ▶ every unit g_j can tune to a different direction $\boldsymbol{\theta}_j$
- ▶ however: **constant rate of change** is too restrictive (!)



SECTION 2.2

Linearity: A Deeper Look

- ▶ A more foundational way to define linearity from **superposition principle**:
- ▶ $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is linear, if

$$f(\alpha \mathbf{x}) = \alpha f(\mathbf{x}) \quad \forall \mathbf{x} \in \mathbb{R}^n, \alpha \in \mathbb{R} \quad (\text{homogeneity})$$

$$\text{and } f(\mathbf{x} + \mathbf{y}) = f(\mathbf{x}) + f(\mathbf{y}) \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n. \quad (\text{additivity})$$

- ▶ f is **linear** iff

$$f(\alpha \mathbf{x} + \beta \mathbf{y}) = \alpha f(\mathbf{x}) + \beta f(\mathbf{y}), \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^n, \quad \forall \alpha, \beta \in \mathbb{R}.$$

- ▶ **Affine functions**: ... not necessarily $f(\mathbf{0}) = \mathbf{0}$

We can link the above abstract definition back to the parameterized view of linear maps via (weight) matrices Θ .

In the Euclidian basis $\{\mathbf{e}_1, \dots, \mathbf{e}_n\}$, a vector $\mathbf{x}$ can (trivially) be written as $\mathbf{x} = \sum_i x_i \mathbf{e}_i$.

This directly leads to the following result.

f is linear $\iff f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ for some unique $\mathbf{A} \in \mathbb{R}^{n \times m}$.

Deep compositional networks rely on the ability to **increase expressivity with compositional depth**.

Composing linear functions does **not** lead to any such gains:

Theorem 1

Let f, g be (matching) linear functions. Then $g \circ f$ is linear.

Deep compositional networks rely on the ability to **increase expressivity with compositional depth**.

Composing linear functions does **not** lead to any such gains:

Theorem 1

Let f, g be (matching) linear functions. Then $g \circ f$ is linear.

Proof.

$$\begin{aligned}(g \circ f)(\alpha \mathbf{x}) &= g(f(\alpha \mathbf{x})) = g(\alpha f(\mathbf{x})) = \alpha g(f(\mathbf{x})) = \alpha (g \circ f)(\mathbf{x}) \\(g \circ f)(\mathbf{x} + \mathbf{y}) &= g(f(\mathbf{x}) + f(\mathbf{y})) = g(f(\mathbf{x})) + g(f(\mathbf{y})) \\&= (g \circ f)(\mathbf{x}) + (g \circ f)(\mathbf{y})\end{aligned}$$



Let $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$, $g(\mathbf{z}) = \mathbf{B}\mathbf{z}$, then

$$(g \circ f)(\mathbf{x}) = \mathbf{C}\mathbf{x}, \quad \mathbf{C} = \mathbf{B}\mathbf{A}$$

.

Composing linear functions amounts to [multiplying matrices](#), the result of which will be yet another matrix.

SECTION 2.3

Linear Autoencoder

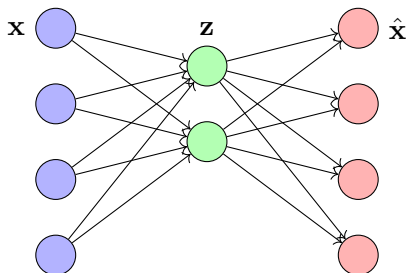
Compress data through a bottleneck layer

$$\mathbf{x} \mapsto \mathbf{z} \mapsto \mathbf{y}, \quad \mathbf{z} = \mathbf{C}\mathbf{x}, \mathbf{y} = \mathbf{D}\mathbf{z}, \quad \mathbf{C}, \mathbf{D}^\top \in \mathbb{R}^{m \times d},$$

where

$$m < d, \quad \ell(\theta) = \frac{1}{2} \|\mathbf{x} - \mathbf{y}\|^2 = \frac{1}{2} \|\mathbf{x} - \mathbf{D}\mathbf{C}\mathbf{x}\|^2 \quad \theta = (\mathbf{C}, \mathbf{D})$$

Autoencoder: Diagram



Goal: minimize $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$

How should you choose C and D to be "optimal"?

Reconstruction Error

Let $\boldsymbol{\theta} = (\mathbf{C}, \mathbf{D})$ and define a loss $L(\boldsymbol{\theta})$ to minimize to reconstruct the input at the output

$$L(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \|\mathbf{x}_i - \mathbf{D}\mathbf{C}\mathbf{x}_i\|^2 \quad (\text{loss})$$

In matrix notation:

$$\mathbf{X} = [\mathbf{x}_1 \dots \mathbf{x}_n] = \begin{pmatrix} | & & | \\ \mathbf{x}_1 & \cdots & \mathbf{x}_n \\ | & & | \end{pmatrix}, \quad \mathbf{Y} = [\mathbf{y}_1 \dots \mathbf{y}_n] = \mathbf{D}\mathbf{C}\mathbf{X}$$

$$L(\boldsymbol{\theta}) = \frac{1}{2} \|\mathbf{X} - \mathbf{Y}\|_F^2 \quad (\text{loss-frobenius})$$

- ▶ Essentially: learning the identity map, $\mathbf{DC} \approx \mathbf{I}$!
- ▶ But (1): trying to learn the identity **with regard to a specific pattern distribution**, represented by a sample
- ▶ But (2): need to find an **efficient lossy compression scheme**

$$\text{Recall } \ell(\theta) = \frac{1}{2} \|\mathbf{x} - \underbrace{\mathbf{D}\mathbf{C}\mathbf{x}}_{=\mathbf{y}}\|^2 = \frac{1}{2} \sum_{j=1}^d (x_j - y_j)^2$$

Stochastic (per datapoint) gradients:

$$\frac{\partial \ell}{\partial (\cdot)} = \sum_{j=1}^d \frac{\partial \ell}{\partial y_j} \frac{\partial y_j}{\partial (\cdot)}.$$

$$\Rightarrow \begin{aligned} \frac{\partial \ell(\theta)}{\partial C_{ki}} &= \sum_{j=1}^d (y_j - x_j) \frac{\partial y_j}{\partial C_{ki}} = x_i \sum_{j=1}^d D_{jk} (y_j - x_j) \\ \frac{\partial \ell(\theta)}{\partial D_{jk}} &= \sum_{i=1}^d (y_i - x_i) \frac{\partial y_i}{\partial D_{jk}} = (y_j - x_j) \sum_{i=1}^d C_{ki} x_i \end{aligned}$$

Alternatively

$$\partial_{\mathbf{A}} f := \left(\frac{\partial f}{\partial A_{ij}} \right)_{ij}$$

(gradient layout)

One can then write elegantly

$$\Rightarrow \begin{aligned} \frac{\partial \ell(\theta)}{\partial \mathbf{C}} &= \mathbf{D}^\top (\mathbf{y} - \mathbf{x}) \mathbf{x}^\top \in \mathbb{R}^{m \times d}, \\ \frac{\partial \ell(\theta)}{\partial \mathbf{D}} &= (\mathbf{y} - \mathbf{x}) \mathbf{x}^\top \mathbf{C}^\top \in \mathbb{R}^{d \times m} \end{aligned}$$

$$\Rightarrow \text{rank}(\mathbf{DC}) \leq \min\{\text{rank}(\mathbf{C}), \text{rank}(\mathbf{D})\} \leq m < d.$$

which implies

$$\Rightarrow \mathbf{Y} = \mathbf{DCX}, \quad \text{rank}(\mathbf{Y}) \leq m$$

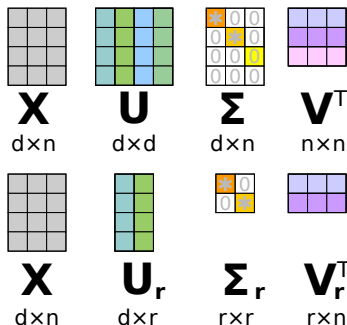
Factorization of a matrix $\mathbf{X} \in \mathbb{R}^{d \times n}$ such that:

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \quad \mathbf{U} \in \mathbb{R}^{d \times d}, \mathbf{V} \in \mathbb{R}^{n \times n}, \text{ orthogonal}, \quad (\text{SVD})$$
$$\mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_{\min\{n, d\}})$$

$$\mathbf{X}_r = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top, \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r},$$

$$\mathbf{U}_r = [\mathbf{u}_1 \dots \mathbf{u}_r], \quad \mathbf{V}_r = [\mathbf{v}_1 \dots \mathbf{v}_r]$$

(reduced SVD)



(Note that various terminologies are used in different ways: thin, compact, reduced)

Theorem 2

The rank- r matrix $\mathbf{X}_r$, obtained from the truncated singular value decomposition is such that:

$$\Rightarrow \|\mathbf{X} - \mathbf{X}_r\|_F = \min_{\text{rank}(\mathbf{Y}) \leq r} \|\mathbf{X} - \mathbf{Y}\|_F \quad (\text{Eckart-Young})$$

How should you choose $\mathbf{C}$ and $\mathbf{D}$ to be "optimal"?

By Theorem 2, choose $\mathbf{C} = \mathbf{U}_m^\top$, $\mathbf{D} = \mathbf{U}_m$:

$$\Rightarrow \boxed{\mathbf{DCX} = \mathbf{X}_m}$$

Proof.

$$\begin{aligned}\mathbf{DCX} &= \mathbf{DCU}\Sigma\mathbf{V}^\top \\ &= \mathbf{U}_m(\mathbf{U}_m^\top\mathbf{U})\Sigma\mathbf{V}^\top \\ &= \mathbf{U}_m\mathbf{I}_m\Sigma\mathbf{V}^\top \\ &= \mathbf{U}_m\Sigma_m\mathbf{V}_m^\top \\ &= \mathbf{X}_m\end{aligned}$$

□

SECTION 3

Non-linear Activations

SECTION 3.1

Ridge Functions

Ridge functions: generalize linear functions in just the right way to allow for powerful compositionality.

Simple idea: compose linear (or affine) function with a non-linear (continuous) function ϕ , also called **activation function**

$$f = \phi \circ g, \quad f(\mathbf{x}; \boldsymbol{\theta}) = \phi(\mathbf{x} \cdot \boldsymbol{\theta}) \quad (\text{ridge unit})$$

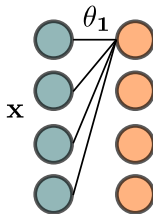
Ridge functions have a **variable rate of change**, governed by ϕ or rather ϕ' (in case ϕ is differentiable).

$$\Rightarrow \boxed{\|\nabla_{\mathbf{x}} f(\mathbf{x}; \boldsymbol{\theta})\| = |\phi'(\mathbf{x} \cdot \boldsymbol{\theta})| \|\boldsymbol{\theta}\|}$$

Ridge units can easily be arranged in layers. We will then write, extending ϕ to vectors,

$$F(\mathbf{x}; \Theta) = \phi(\Theta \mathbf{x}), \quad \phi(\mathbf{z}) = \begin{pmatrix} \phi(z_1) \\ \dots \\ \phi(z_m) \end{pmatrix}$$

(ridge layer)



Parameter update directions for ridge functions are always in the **direction of the input vector**.

For any loss function ℓ , the chain rule yields

$$\Rightarrow \nabla_{\boldsymbol{\theta}}(\ell \circ \phi)(\mathbf{x} \cdot \boldsymbol{\theta}) = (\ell \circ \phi)'(\mathbf{x} \cdot \boldsymbol{\theta}) \mathbf{x}.$$

This is an inheritance of the way ridge functions generalize linear functions.

SECTION 3.2

Threshold Units

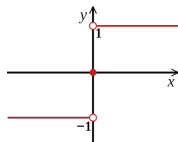
Heavyside or Sign Units

A **linear threshold unit** is a function that uses the Heavyside or sign function as activation and produces Boolean outputs.

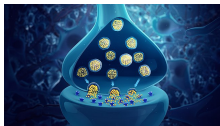
$$f^H(\mathbf{x}; \boldsymbol{\theta}) = H(\mathbf{x} \cdot \boldsymbol{\theta}), \quad H(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{else} \end{cases} \quad (\text{threshold unit})$$

Equivalently, one can also map to $\{-1, 1\}$,

$$f^{\pm}(\mathbf{x}; \boldsymbol{\theta}) = \text{sign}(\mathbf{x} \cdot \boldsymbol{\theta}). \quad (\text{sign unit})$$



Threshold units = **cartoon version of a biological neuron**.



⇒ If activation exceeds a threshold, the unit 'fires'.

Major drawback: H has a jump at 0

⇒ no derivative information to guide the adaptation of the parameters

⇒ rules out gradient-based methods.

SECTION 3.3

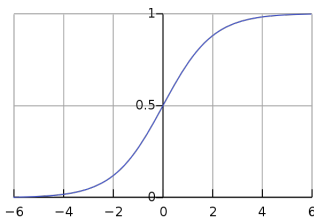
Sigmoid Units

Goal: retain some characteristics of the threshold unit, in particular the limits $f(\mathbf{x}, \boldsymbol{\theta}) \rightarrow (0, 1)$ as $\mathbf{x} \cdot \boldsymbol{\theta} \rightarrow (-\infty, \infty)$, but to **define a smooth, monotone function**.

One such choice is the **logistic Unit**

$$\sigma(\mathbf{x} \cdot \boldsymbol{\theta}) = \frac{1}{1 + \exp[-\mathbf{x} \cdot \boldsymbol{\theta}]}.$$

(logistic unit)



Proposition 3

The logistic function has derivative

$$\Rightarrow \sigma'(z) = \sigma(z)(1 - \sigma(z)) = \sigma(z)\sigma(-z).$$

Proof: With the quotient rule,

$$\begin{aligned}\sigma'(z) &= e^{-z}(1 + e^{-z})^{-2} = \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}} \\ &= \sigma(z)(1 - \sigma(z)).\end{aligned}$$

This implies that all higher-order derivatives are polynomials in σ and hence σ is smooth:

$$\Rightarrow \sigma \in C^\infty \tag{1}$$

Another popular choice of activation function is the hyperbolic tangent, which is **simple transformation of the logistic function**

$$\Rightarrow \tanh(z) := \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1,$$

where the identity follows from:

$$2\sigma(2z) - 1 = \frac{2}{1 + e^{-2z}} - \frac{1 + e^{-2z}}{1 + e^{-2z}} = \frac{1 - e^{-2z}}{1 + e^{-2z}} = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

As the range of the $\tanh$ function $R = (-1; 1)$ is symmetric around zero, it is sometimes preferred.

Finally, we also derive the inverse of the logistic function for later use

$$\Rightarrow \boxed{\sigma^{-1}(t) = \ln \frac{t}{1-t}}, \quad (\text{inverse logistic})$$

The validity of this formula can be checked via:

$$\sigma \left(\ln \frac{t}{1-t} \right) = \frac{1}{1 + \frac{1-t}{t}} = t.$$

SECTION 3.4

Softmax

A generalization of logistic units for **multiclass classification** (k classes).

Defines class conditional probabilities via

$$\sigma_i^{\max}(\mathbf{x}; \Theta) = \frac{\exp[\mathbf{x} \cdot \boldsymbol{\theta}_i]}{\sum_{j=1}^k \exp[\mathbf{x} \cdot \boldsymbol{\theta}_j]}, \quad \Theta = [\boldsymbol{\theta}_1, \dots, \boldsymbol{\theta}_k] \quad (\text{softmax})$$

Properties:

- ▶ Non-negative
- ▶ Normalized: $\sum_{i=1}^k \sigma_i^{\max}(\mathbf{x}; \Theta) = 1$

Softmax recovers sigmoid for $k = 2$

For $k = 2$, we get

$$\Rightarrow \boxed{\text{soft-max with } \boldsymbol{\theta}_1, \boldsymbol{\theta}_2 \iff \text{sigmoid with } \boldsymbol{\theta} = \boldsymbol{\theta}_1 - \boldsymbol{\theta}_2}$$

$$\begin{aligned}\sigma_1^{\max}(\mathbf{x}) &= \frac{e^{\mathbf{x} \cdot \boldsymbol{\theta}_1}}{e^{\mathbf{x} \cdot \boldsymbol{\theta}_1} + e^{\mathbf{x} \cdot \boldsymbol{\theta}_2}} = \frac{e^{\mathbf{x} \cdot (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_1)}}{e^{\mathbf{x} \cdot (\boldsymbol{\theta}_1 - \boldsymbol{\theta}_1)} + e^{\mathbf{x} \cdot (\boldsymbol{\theta}_2 - \boldsymbol{\theta}_1)}} \\ &= \frac{1}{1 + e^{-\mathbf{x} \cdot \boldsymbol{\theta}}} = \sigma(\mathbf{x})\end{aligned}$$

The softmax function is invariant under adding the same constant to all logits

↪ Why? because subtracting the same constant from every logit cancels out between the numerator and denominator.

Common trick Subtract the maximum logit before applying the exponential function. Given logits $z_1, \dots, z_C$, compute

$$\text{softmax}(z_i) = \frac{\exp(z_i - \max_j z_j)}{\sum_{k=1}^C \exp(z_k - \max_j z_j)} = \frac{\exp(z_i)}{\sum_{k=1}^C \exp(z_k)}.$$

Why does this help? $\exp(z_i)$ can overflow when some logits are large. After subtracting the maximum logit, the largest exponent is always $\exp(0) = 1 \Rightarrow$ computation much more numerically stable.

Softmax is typically coupled with cross entropy loss.

Encode class information in one-hot vector, $\mathbf{y} \in \{\mathbf{e}_1, \dots, \mathbf{e}_k\}$, then:

$$\ell(\mathbf{x}, \mathbf{y}; \Theta) = -\mathbf{y} \cdot \ln \sigma^{\max}(\mathbf{x}; \Theta)$$

Alternatively: separate out the normalization

$$\ell(\mathbf{x}, \mathbf{y}; \Theta) = - \sum_{i=1}^k y_i \mathbf{x} \cdot \boldsymbol{\theta}_i + \ln \left(\sum_{i=1}^k \exp[\mathbf{x} \cdot \boldsymbol{\theta}_i] \right) .$$

leading to the gradient formula

$$\Rightarrow \nabla_{\boldsymbol{\theta}_i} \ell(\mathbf{x}, \mathbf{y}; \Theta) = (\sigma_i^{\max} - y_i) \mathbf{x}$$

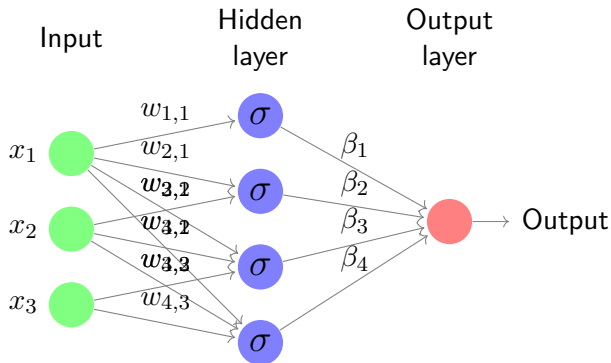
SECTION 3.5

Multilayer Perceptron

- ▶ Three-layer reference architecture with
 - n inputs
 - one hidden layer of m sigmoid units
 - one real-valued output (could be more)

$$f(\mathbf{x}; \boldsymbol{\beta}, \mathbf{w}) = \sum_{j=1}^m \beta_j \sigma(\mathbf{w}_j \cdot \mathbf{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- ▶ Each unit computes a weighted combination of inputs and applies a logistic activation function.
- ▶ These activations (in range $(0; 1)$) are then summed up with weights $\boldsymbol{\beta}$.



Use the following squared loss to learn the weights:

$$\mathcal{L}(\boldsymbol{\beta}, \mathbf{w}; \mathbf{x}, y) = \frac{1}{2} (f(\mathbf{x}) - y)^2.$$

By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial \theta} = (f(\mathbf{x}) - y) \frac{\partial f(\mathbf{x})}{\partial \theta} \quad \text{for any parameter } \theta.$$

$$\frac{\partial \frac{1}{2} (f(\mathbf{x}) - y)^2}{\partial \beta_j} = \frac{f(\mathbf{x}) - y}{1 + \exp[-\mathbf{w}_j \cdot \mathbf{x}]},$$

$$\frac{\partial \frac{1}{2} (f(\mathbf{x}) - y)^2}{\partial w_{ji}} = \frac{f(\mathbf{x}) - y}{1 + \exp[-\mathbf{w}_j \cdot \mathbf{x}]} \cdot \frac{\beta_j x_i}{1 + \exp[\mathbf{w}_j \cdot \mathbf{x}]}.$$

Learning the weights

- ▶ Forward pass: computes hidden activities and output
- ▶ Re-combined with a few more operations to get derivatives
- ▶ Updates via stochastic gradient descent (SGD)

$$\theta \leftarrow \theta - \eta \frac{\mathcal{L}(\boldsymbol{\beta}, \mathbf{w}; \mathbf{x}, y)}{\partial \theta}, \quad \theta \in \{\beta_j, w_{ji}\},$$

with some step size $\eta > 0$.

```
from scipy.special import expit #sigmoid
class MLP3():
    def __init__(self,n,m):
        self.theta
            = np.random.uniform(-1/n,1/n,(m,n)) # mxn
        self.beta
            = np.random.uniform(-1/m,1/m,(1,m)) # 1xm

    def forward(self, x):
        net_in    = np.dot(self.theta,x)           # mxn*nx=mxs
        f['hid']  = expit(net_in)                   # mxs
        f['out']  = np.dot(self.beta,f['hid'])      # 1xm*mxs=1xs
        return f
```

```
def gradient(self, f, x, y):

    # beta gradients
    delta      = (f['out']-y)                # 1xs
    g['bet']   = np.dot(delta,f['hid'].T)     # 1xs*sxm=1xm

    # theta gradients
    hid        = f['hid']                    # hidden activities
    hid_sv     = hid-hid*hid                 # sensitivities
    outer      = np.outer(self.beta.T,delta)# mx1*1xs=mxs
    g['tet']   = np.dot((outer*hid_sv),x.T)  # mxs*sxn=mxn

    return g
```

SECTION 3.6

Backpropagation

- ▶ Backpropagation is a supervised learning algorithm used for training artificial neural networks.
- ▶ It calculates the gradient of the loss function with respect to each weight by the chain rule, iterating backward from the output to the input layer.
- ▶ The main goal is to minimize the error by adjusting the weights.

Example Network Structure

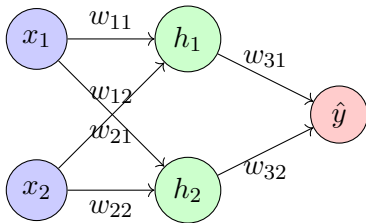


Figure: A simple neural network with input layer, one hidden layer, and output layer.

- Calculate the input to the hidden layer neurons:

$$\text{net}_{h_1} = w_{11}x_1 + w_{21}x_2$$

$$\text{net}_{h_2} = w_{12}x_1 + w_{22}x_2$$

- Calculate the output of the hidden layer neurons using an activation function σ :

$$h_1 = \sigma(\text{net}_{h_1})$$

$$h_2 = \sigma(\text{net}_{h_2})$$

- Calculate the input to the output neuron:

$$\text{net}_{o1} = w_{31}h_1 + w_{32}h_2$$

- Calculate the output neuron:

$$\hat{y} = \sigma(\text{net}_{o1})$$

- Compute the loss (error) using a loss function, e.g., Mean Squared Error (MSE):

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

- Here, y is the true label, and $\hat{y}$ is the predicted output.

We start by computing the gradient of the loss with respect to the output:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

Gradient with Respect to Weights (Hidden to Output)

Recall that

$$\hat{y} = \sigma(\text{net}_{o1}) \quad \text{net}_{o1} = w_{31}h_1 + w_{32}h_2.$$

Then, we compute the gradient of the loss with respect to the weights connecting the hidden layer to the output layer.

$$\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{net}_{o1}}{\partial w_{31}} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot h_1$$

$$\frac{\partial L}{\partial w_{32}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{net}_{o1}}{\partial w_{32}} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot h_2$$

The next step is to compute the gradient of the loss with respect to the hidden layer neurons:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{net}_{o1}}{\partial h_1} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot w_{31}$$

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \text{net}_{o1}} \cdot \frac{\partial \text{net}_{o1}}{\partial h_2} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot w_{32}$$

Illustration Backpropagation

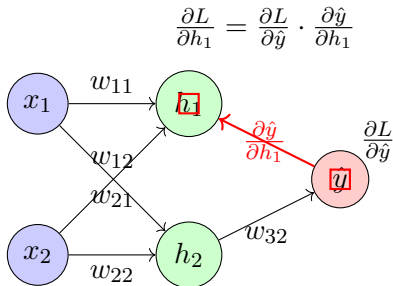


Figure: Illustration of the backward pass in the neural network. The red gradients are backpropagated gradients.

Gradient with Respect to Weights (Input to Hidden)

We then compute the gradient of the loss with respect to the weights connecting the input layer to the hidden layer. This also involves the derivative of the activation function σ applied to net_{h_1} and net_{h_2} :

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial \text{net}_{h_1}} \cdot \frac{\partial \text{net}_{h_1}}{\partial w_{11}} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot w_{31} \cdot \sigma'(\text{net}_{h_1}) \cdot x_1$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial \text{net}_{h_1}} \cdot \frac{\partial \text{net}_{h_1}}{\partial w_{21}} = (\hat{y} - y) \cdot \sigma'(\text{net}_{o1}) \cdot w_{31} \cdot \sigma'(\text{net}_{h_1}) \cdot x_2$$

- Update the weights using gradient descent:

$$w_{ij} := w_{ij} - \eta \cdot \frac{\partial L}{\partial w_{ij}}$$

- Here, η is the learning rate.
- Repeat the forward and backward passes for multiple iterations (epochs) until the loss converges.